



UE23CS352B - Object Oriented Analysis & Design

Mini Project Report

Title

SUBSYSTEM 11: BARCODE READER & RFID TRACKER

Submitted by:

Team Members

NAME	SRN
Abhikruti GNS	PES1UG23AM012
Adithi Prabhu	PES1UG23AM018
Anusha	PES1UG23AM060
Ananya J C	PES1UG23AM046

Semester - Section
6TH SEM - A SECTION

Faculty Name

Prof. Anand M.S.

January - May 2026

DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING (AI & ML)

PES UNIVERSITY

(Established under
Karnataka Act No. 16 of
2013) 100ft Ring Road,
Bengaluru – 560 085,
Karnataka, India

Problem Statement:

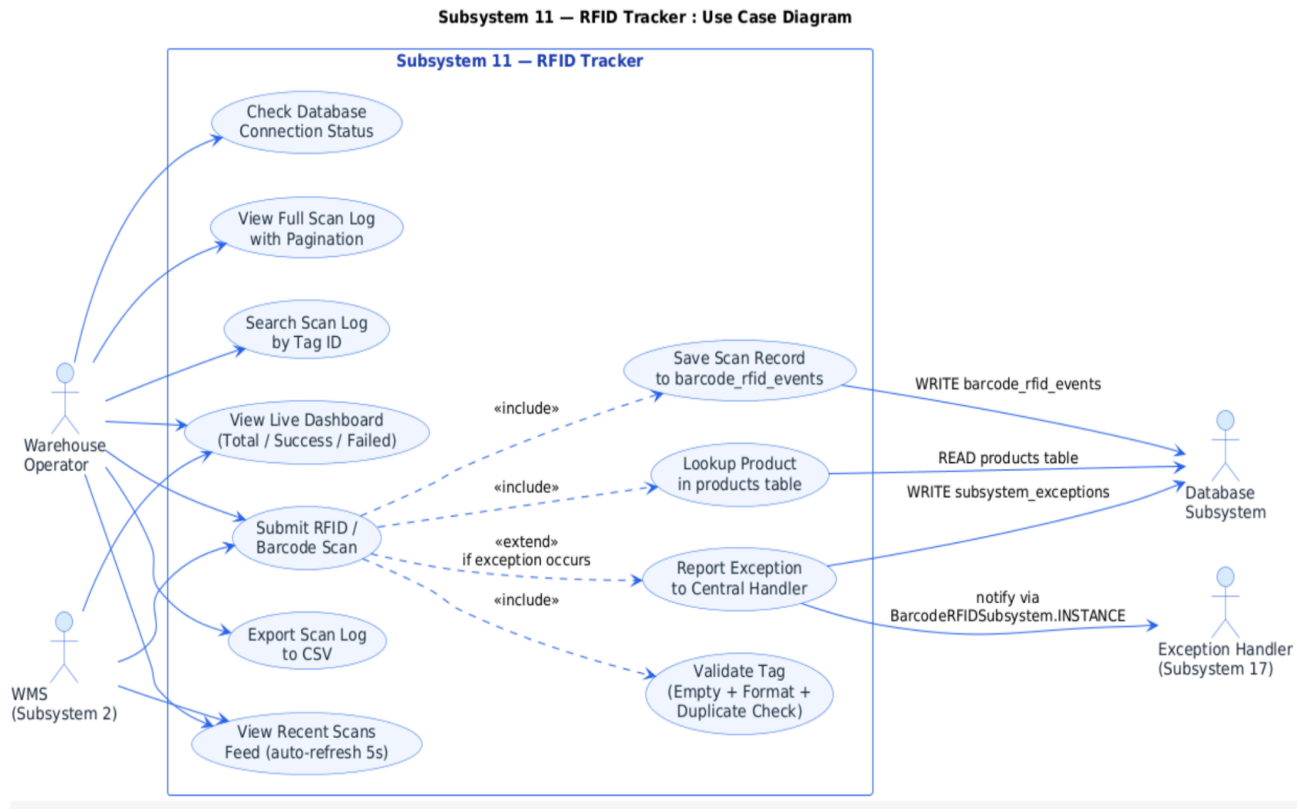
Subsystem 11 is the RFID tracker module of a larger supply chain management (SCM) system. The problem being solved is the need for a reliable, real-time system to capture and record product arrivals at a warehouse using RFID tags and barcodes. Without this subsystem, there is no automated way to know when a product has arrived, whether it has been scanned before, or whether the tag is even valid. The subsystem acts as the first point of contact in the warehouse, every product that arrives must pass through this system before inventory is updated or any downstream processing begins. The system must validate incoming tags, match them to known products, record scan events, detect duplicates, update inventory stock counts, and report exceptions .

Key Features:

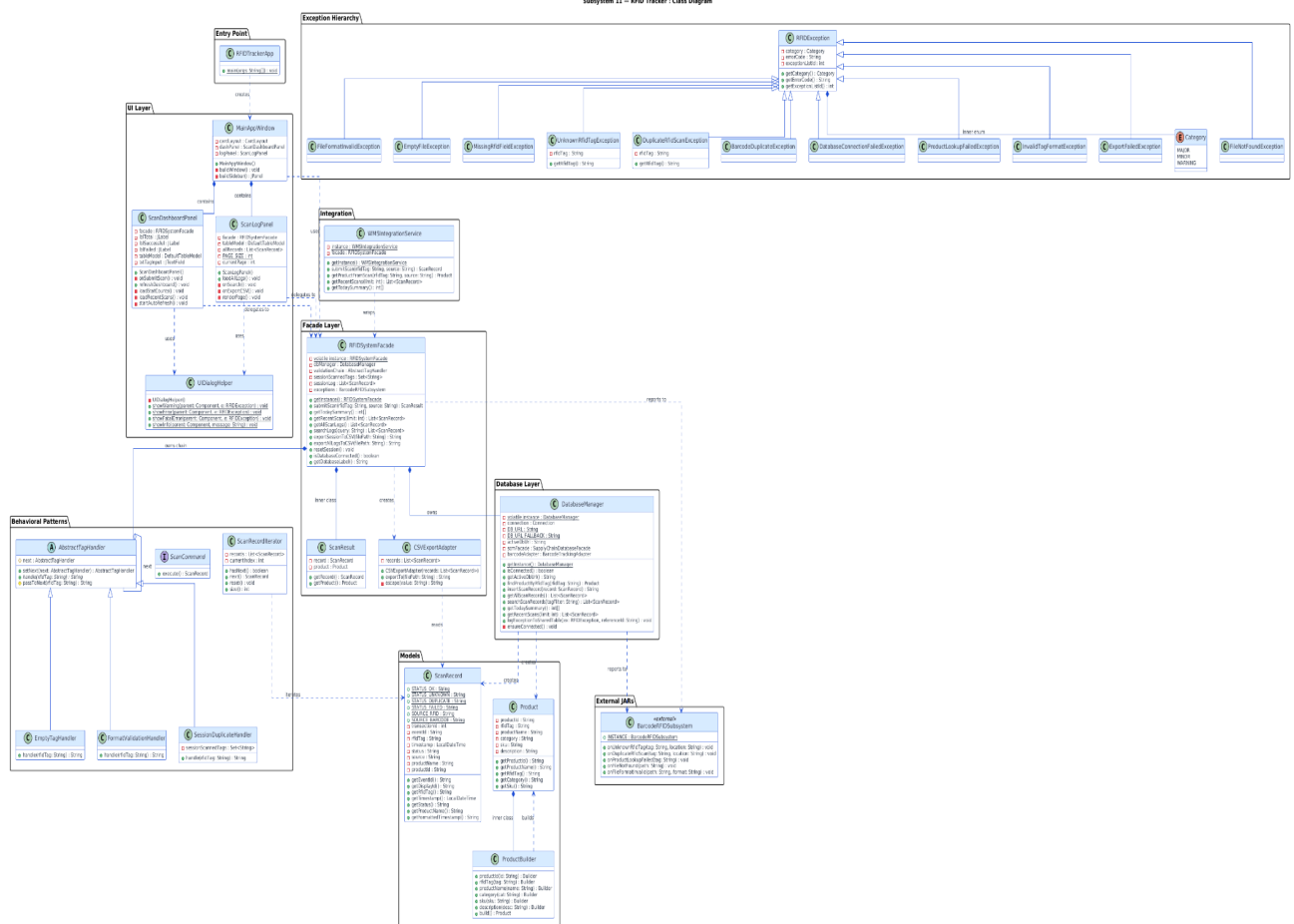
- Manual RFID / Barcode Scan Submission
- Real-Time Stat Dashboard (Total / Success / Failed)
- Live Recent Scans Feed (auto-refresh every 5s)
- Duplicate Scan Detection (session-level)
- Scan Log
- Search / Filter Scan Log by Tag ID
- CSV Export of Scan Log

Models:

Use Case Diagram:

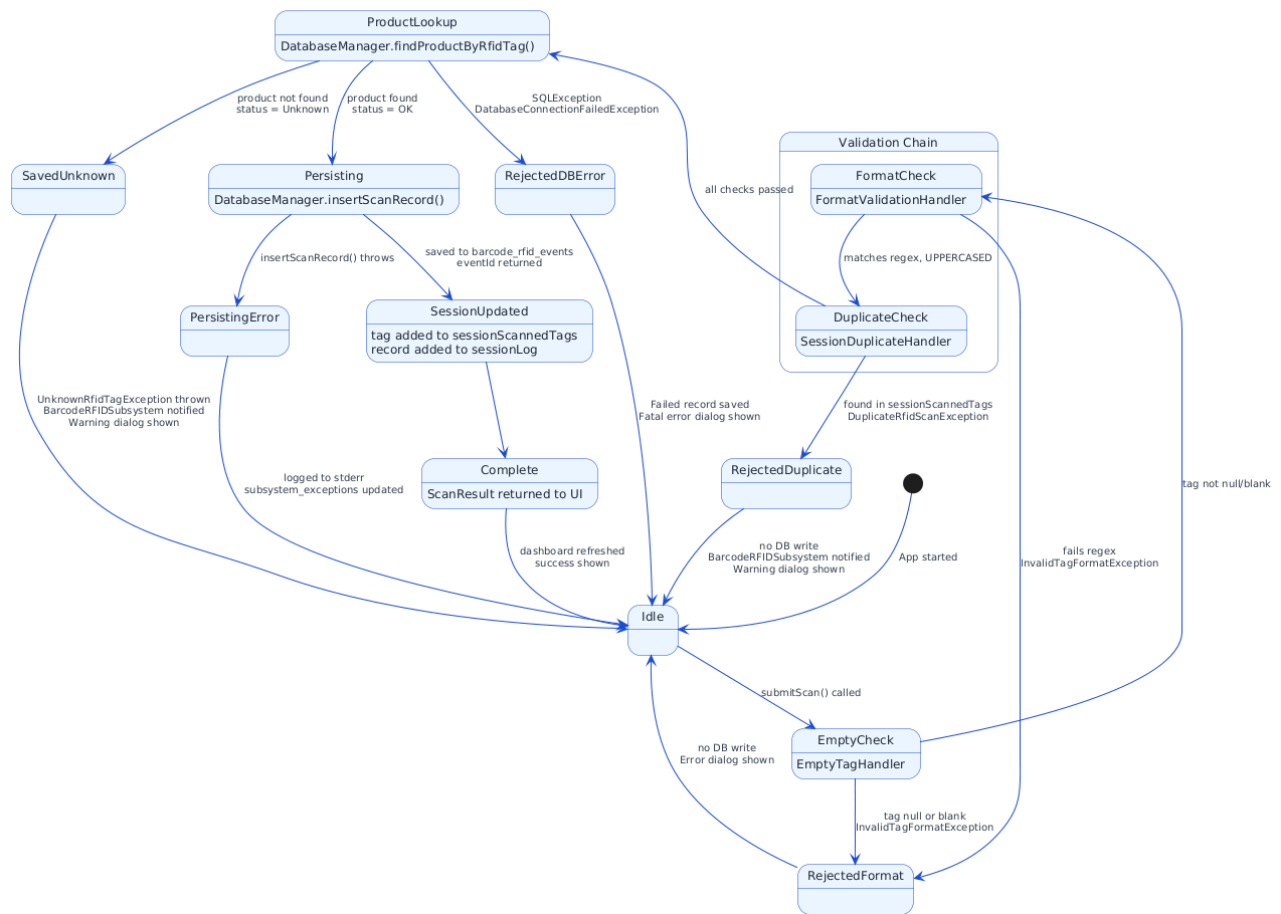


Class Diagram:



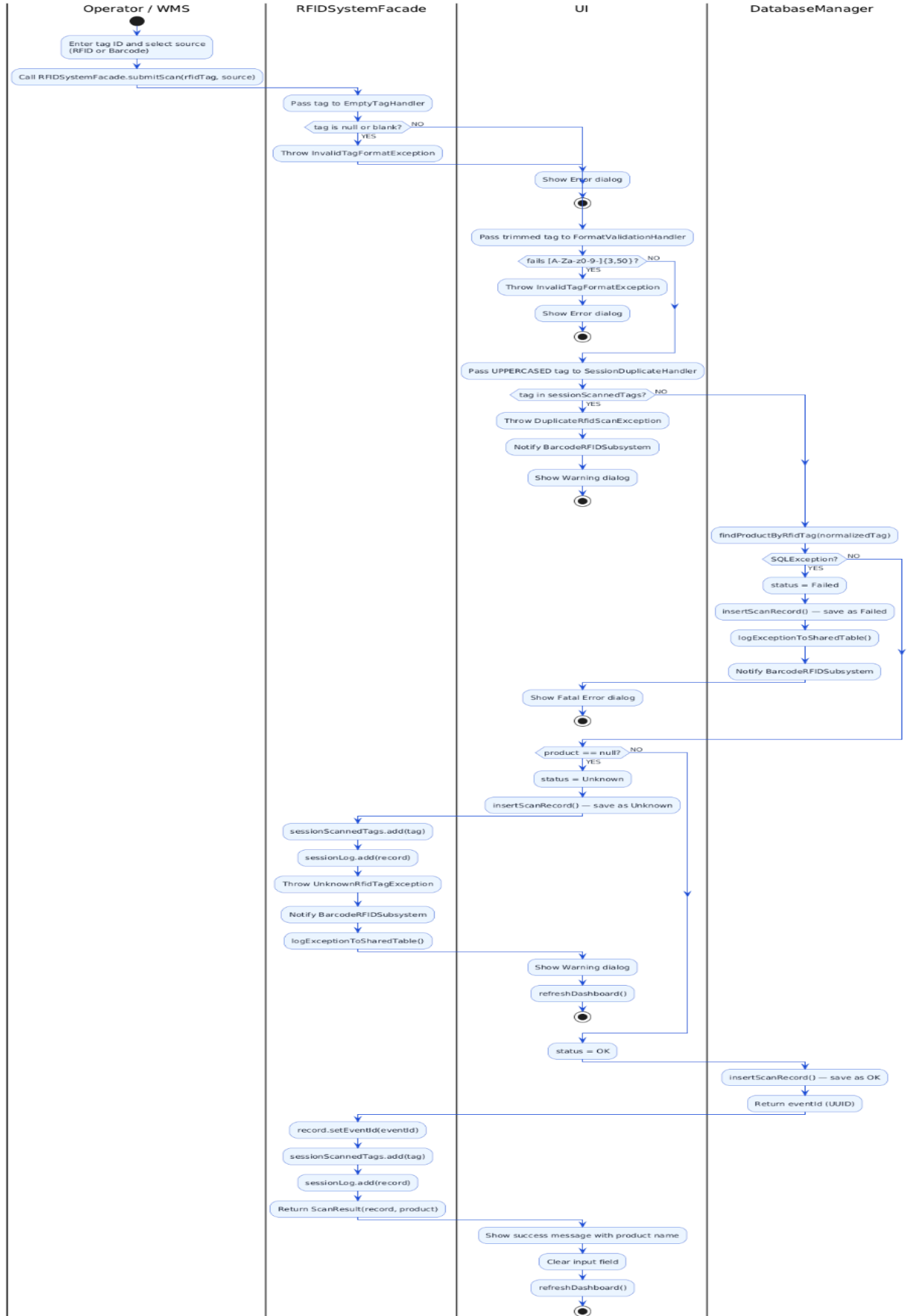
State Diagram:

Subsystem 11 — RFID Tracker : State Diagram



Activity Diagrams:

Subsystem 11 — RFID Tracker : Activity Diagram (submitScan)



Design Principles and Design Patterns:

Is MVC architecture used? Yes/No

Yes, the project follows a layered architecture that maps to MVC. The **model** layer consists of the data classes Product and ScanRecord, which represent the core domain objects. The **View** layer is the UI package MainAppWindow, ScanDashboardPanel, ScanLogPanel, and UIDialogHelper, which handle all rendering and user interaction. The **controller** layer is RFIDSystemFacade, which receives requests from the view, coordinates the model and database layers, and returns results. The View never directly accesses the database or business logic;

Note: it always goes through the Facade, maintaining a clean separation between presentation and logic.

Design Principles

SOLID Principles

SRP: Single Responsibility Principle

Every class in the system has exactly one reason to change. DatabaseManager is only responsible for database connections and queries. It does not validate tags or render UI.

CSVExportAdapter is only responsible for writing scan records to a CSV file.

UIDialogHelper is only responsible for showing styled popup dialogs.

ScanDashboardPanel is only responsible for rendering the dashboard screen.

RFIDException is only responsible for defining the error taxonomy of the subsystem. If the database schema changes, only DatabaseManager needs to change. If the dialog styling changes, only UIDialogHelper needs to change.

OCP: Open/Closed Principle

It is visible in the validation chain. AbstractTagHandler defines the contract, each handler receives a tag, validates one rule, and either throws an exception or passes it forward to log the rfid.

Currently there are three handlers: Empty Tag Handler (Makes sure any empty tag doesn't get logged), Format Validation Handler (validates the format prescribed: any alphanumeric value without any special characters is a valid tag this decision was made to keep the subsystem flexible and the tag must be of length 3-50 characters), and

Session Duplicate Handler (Catches all duplicate tags in the session). Adding a new validation rule requires only writing a new subclass and inserting it into the chain in RFIDSystemFacade's constructor. No existing handler is modified.

The same principle applies to the export system, CSVExportAdapter handles CSV export. Adding Excel or JSON export requires only a new Adapter class with no changes to RFIDSystemFacade.

The exception hierarchy also follows OCP, new exception types are added as new inner static classes of RFIDException without touching existing exception handling code.

LSP : Liskov Substitution Principle

All three concrete handlers: EmptyTagHandler, FormatValidationHandler, and SessionDuplicateHandler are valid substitutes for AbstractTagHandler.

Each honors the same contract: given a tag string, either return the validated tag or throw an RFIDException.

The RFIDSystemFacade builds and uses the chain through AbstractTagHandler references only. It never casts or checks the specific type. Similarly, all subclasses of RFIDException are valid substitutes wherever RFIDException is caught. The persistRecord() method in the facade catches RFIDException broadly and handles all subtypes correctly without needing to know which specific subtype it received.

ISP: Interface Segregation Principle

Interface Segregation means no class should be forced to know about methods it doesn't need. UI Scan Page only sees → submitScan() ,Reports only sees → getScanLogs(), getTodaySummary(), getRecentScans() ,UI Dashboard only sees → getTodaySummary(), getRecentScans(),UI Export Page only sees → exportSessionToCSV(), exportAllLogsToCSV(). None of them ever touches ,DatabaseManager directly, TagValidationHandlers directly . So even though the facade has 10+ methods, each caller only uses the 1-3 methods relevant to them.

DIP: Dependency Inversion Principle

RFIDSystemFacade depends on DatabaseManager as an abstraction over raw JDBC.

The facade never calls DriverManager.getConnection() or writes SQL directly. All database interaction is delegated to DatabaseManager.

High-level coordination logic in the facade is insulated from low-level database and file I/O details.

Design Patterns

Creational Patterns

Singleton: DatabaseManager and RFIDSystemFacade

DatabaseManager uses the Singleton pattern with double-checked locking and a volatile instance variable to ensure exactly one database connection exists across the entire application. Multiple connections would result in inconsistent state.

RFIDSystemFacade is also a Singleton because it owns the sessionScannedTags HashSet used for duplicate detection. If two Facade instances existed, they would have separate HashSets and duplicate detection would fail. Both singletons are thread-safe via the synchronized block in getInstance().

Builder: Product

Product uses the Builder pattern because it has six fields and is constructed from two different database schemas. The shared OOAD database uses VARCHAR product IDs and SKU columns, while the legacy database uses integer product IDs and an rfid_tag column. The Builder allows named field assignment regardless of schema, avoids constructor parameter confusion, and produces an immutable object once build() is called.

Structural Patterns

Facade: RFIDSystemFacade

RFIDSystemFacade is the single entry point to the entire subsystem. Without it, the UI would need to be directly coupled to DatabaseManager, AbstractTagHandler, CSVExportAdapter, and BarcodeRFIDSubsystem, five separate dependencies. With the facade, the UI has exactly one dependency. Other subsystems (WMS, Reporting) also interact only through the Facade's public API.

The Facade coordinates the scan workflow: validation → product lookup →

persistence → session tracking → inventory notification → exception reporting.

Adapter: CSVExportAdapter

CSVExportAdapter adapts an internal List<ScanRecord> to a file export operation. The Facade calls adapter.exportTo(path) and receives back the saved file path, it has no knowledge of PrintWriter, FileWriter, or CSV formatting. The Adapter pattern ensures that adding a new export format (Excel, JSON) requires only writing a new Adapter class. No changes to the Facade or UI are needed, satisfying OCP.

Behavioural Patterns

Chain of Responsibility: Tag Validation

Tag validation is implemented as a chain of three independent handlers. EmptyTagHandler checks for null or blank input. FormatValidationHandler checks that the tag matches the alphanumeric regex pattern and is between 3 and 50 characters, and normalises it to uppercase. SessionDuplicateHandler checks the session-level HashSet for previously scanned tags. Each handler either throws a specific RFIDException subclass or passes the tag to the next handler via passToNext(). The chain is assembled in RFIDSystemFacade's constructor and is reused for every scan.

Command: ScanCommand

ScanCommand is a single-method interface with execute(). Any scan action can be encapsulated as a command object and executed uniformly. This decouples the action from its caller and allows scan operations to be queued, logged, or undone if needed in future extensions.

Iterator: ScanRecordIterator

ScanRecordIterator provides sequential access to a list of ScanRecord objects without exposing the underlying list structure. The ScanLogPanel also applies the Iterator concept at the UI level renderPage() iterates a sublist of allRecords using index-based slicing, traversing records page by page without the UI needing to know how records are stored internally.

GRASP Principles

Information Expert

The class that has the information is given the responsibility to work with it. ScanRecord knows all details about one scan event tag, timestamp, status, source, product name so it provides getFormattedTimestamp() and getDisplayId() itself. Product owns its own fields and provides getProductIdInt() for legacy conversion. DatabaseManager owns all knowledge of how to talk to the database, so all SQL queries live inside it. No other class reaches into another class's data directly.

Creator

RFIDSystemFacade creates ScanRecord objects because it has all the data needed to initialise one the tag, the timestamp, the status, the source, and the product. This follows the GRASP Creator rule: assign creation responsibility to the class that has the initialising data.

Controller

RFIDSystemFacade is the GRASP Controller. It is a non-UI class that handles all system operations: scan submission, log retrieval, CSV export, session reset. The UI panels (ScanDashboardPanel, ScanLogPanel) never implement business logic themselves. They only capture user events and delegate to the Facade. This keeps the UI layer purely presentational.

Low Coupling

ScanDashboardPanel and ScanLogPanel are coupled only to RFIDSystemFacade. They do not know about DatabaseManager, CSVExportAdapter, or BarcodeRFIDSubsystem. If any of these internal classes change, the UI is completely unaffected. Other subsystems like WMS and Reporting are also coupled only to the Facade's public API.

High Cohesion

Each class has a narrow, focused set of responsibilities. DatabaseManager groups all database operations. RFIDException groups all exception type definitions. UIDialogHelper groups all dialog display logic. The CSVExportAdapter groups all CSV writing logic. No class mixes unrelated concerns.

Polymorphism

AbstractTagHandler defines an abstract handle() method. The three concrete handlers EmptyTagHandler, FormatValidationHandler, and SessionDuplicateHandler each override it with their specific validation logic. The facade calls handle() on the chain without knowing which specific handler is executing at any point. This is polymorphic dispatch.

Indirection

RFIDSystemFacade introduces indirection between the UI and all internal subsystem components. The UI never directly references DatabaseManager or the validation chain. BarcodeRFIDSubsystem. INSTANCE introduces indirection between the RFID subsystem and the central exception handler, neither side knows the other's implementation details.

Protected Variations

The dual-database fallback logic is entirely hidden inside DatabaseManager. Whether the system is connected to the shared OOAD database or the local fallback scm_rfid_db, all other classes call the exact same DatabaseManager methods. The variation point is isolated. Similarly, CSVExportAdapter hides all file I/O variation from the facade, the export format can change without the facade knowing.

Pure Fabrication

UIDialogHelper does not represent any real-world warehouse concept. It was invented purely to centralize dialog styling and avoid code duplication across ScanDashboardPanel and ScanLogPanel. CSVExportAdapter is also a pure fabrication; there is no real-world domain object called a "CSV Exporter." Both classes exist to achieve low coupling and high cohesion.

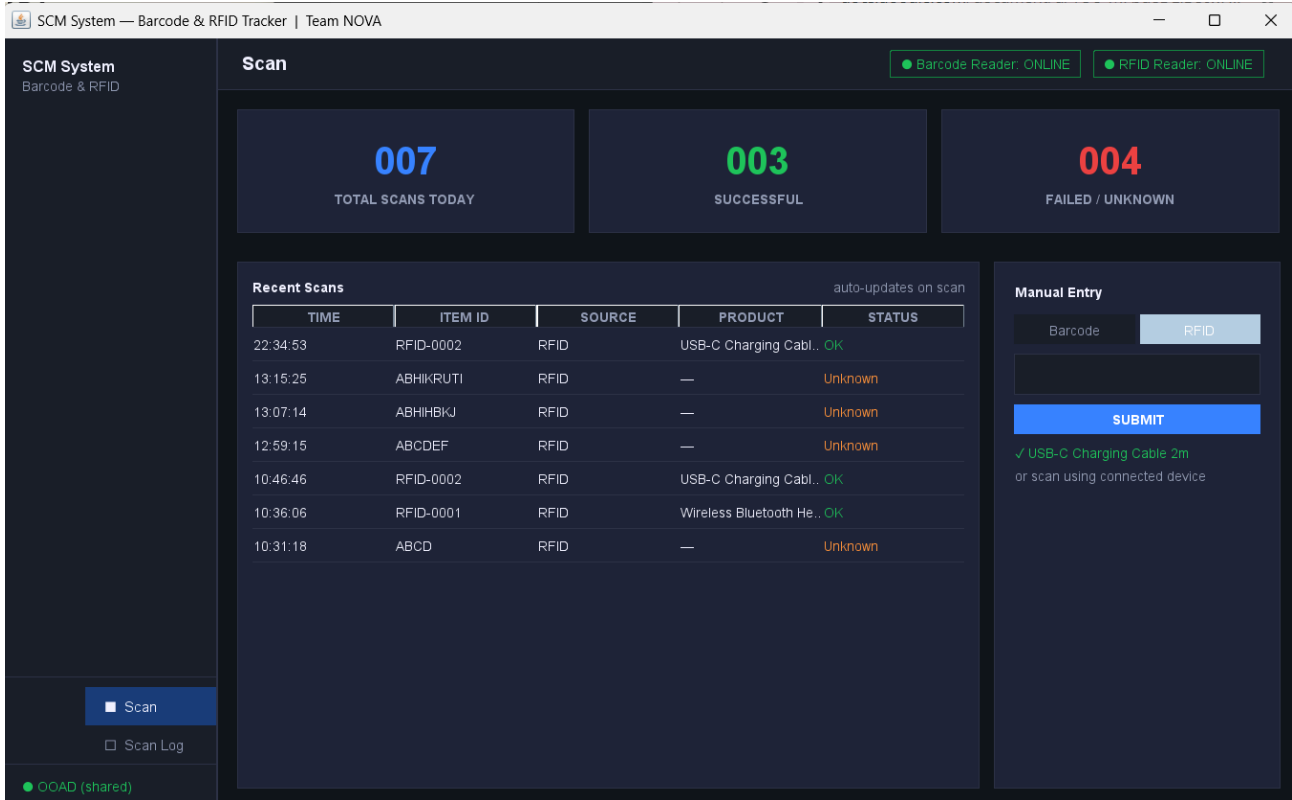
Github link to the Codebase: (repository should be public)

https://github.com/OOAD-Section-A/rfid_tracker.git

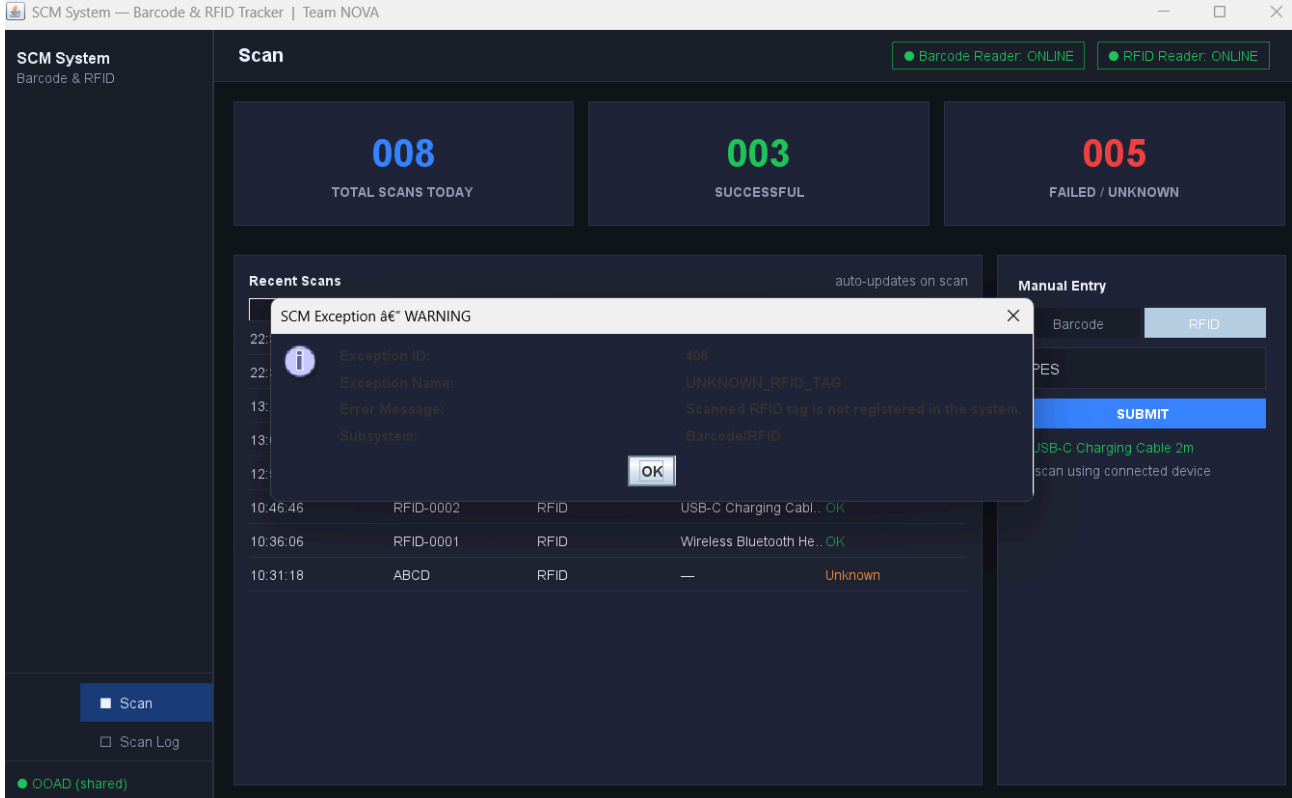
Screenshots

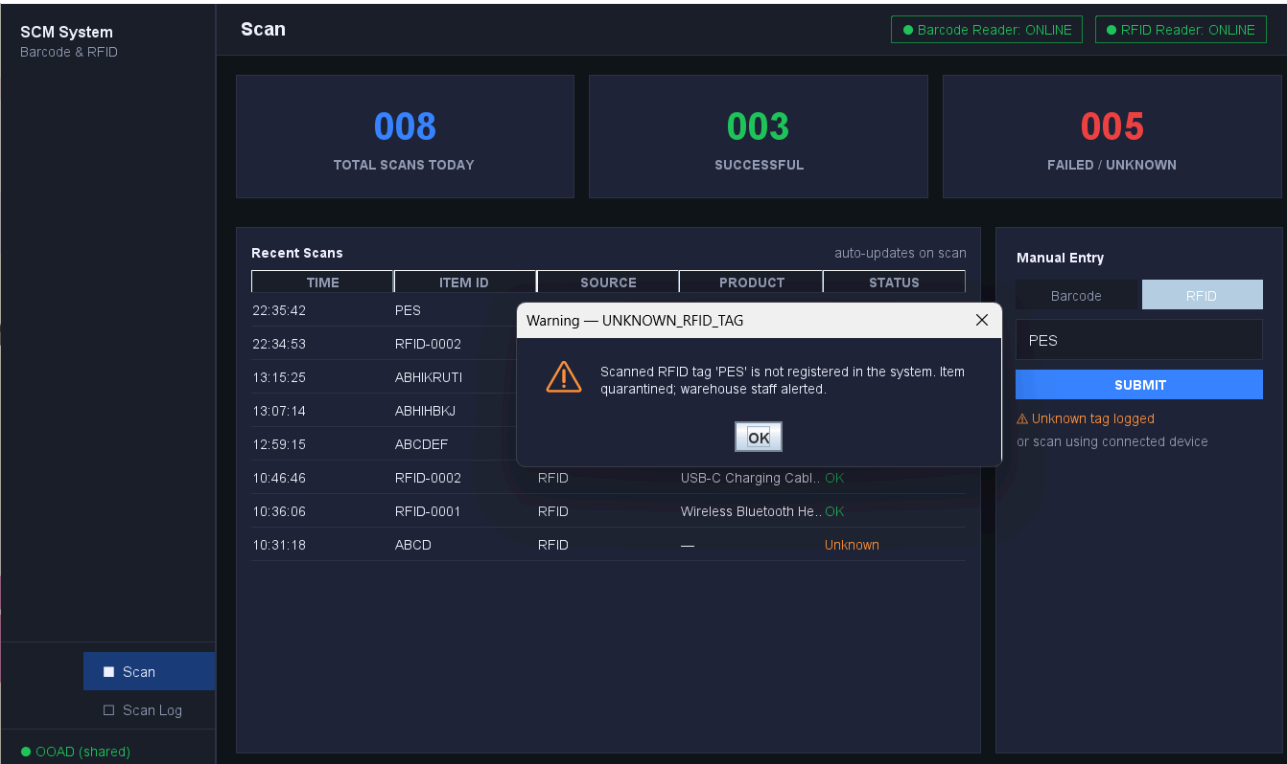
UI:

Dashboard

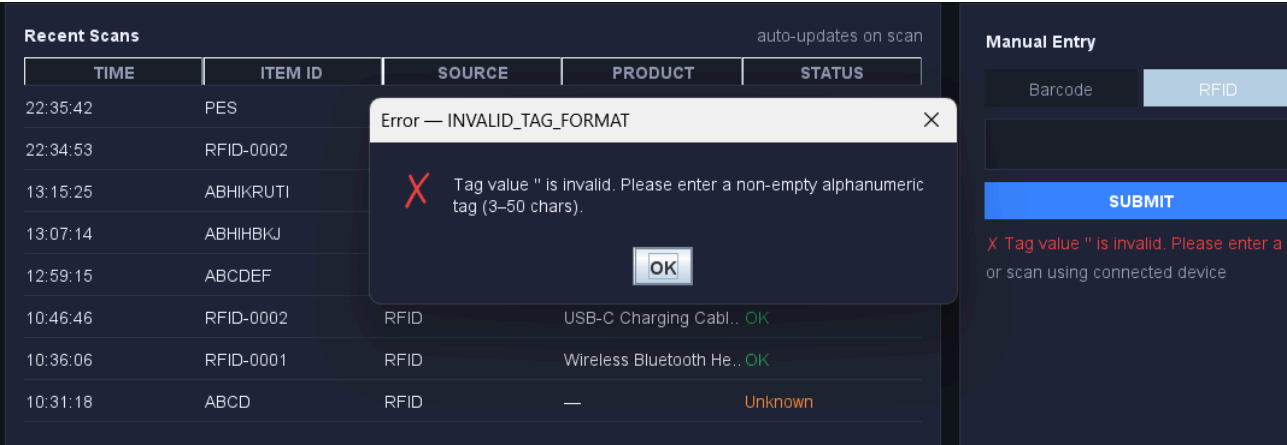


Unknown Tag:

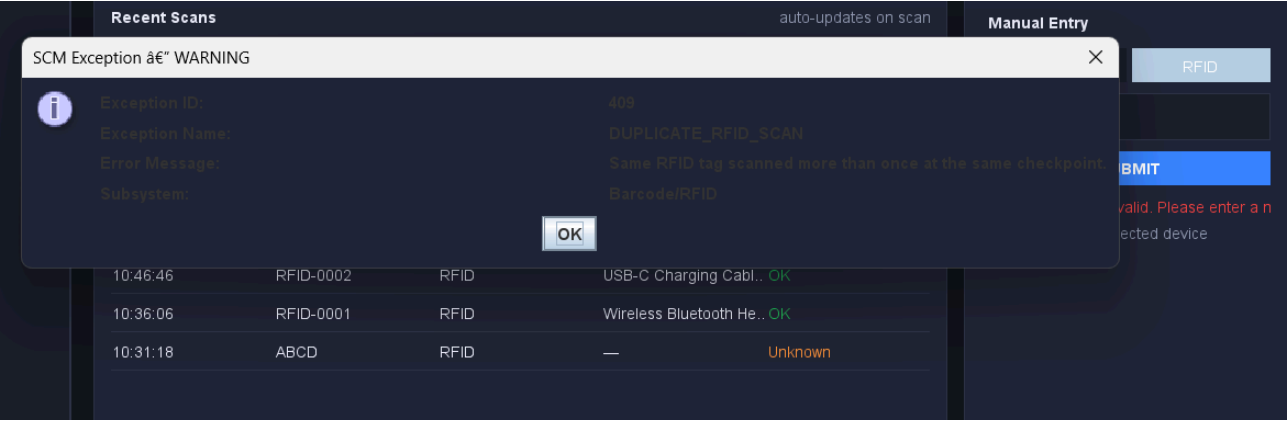




Invalid Tag format:



Duplicate tag:



Recent Scans

auto-updates on scan

TIME	ITEM ID	SOURCE	PRODUCT	STATUS
22:35:42	PES			
22:34:53	RFID-0002			
13:15:25	ABHIKRUTI			
13:07:14	ABHIHBKJ			
12:59:15	ABCDEF			
10:46:46	RFID-0002	RFID	USB-C Charging Cabl..	OK
10:36:06	RFID-0001	RFID	Wireless Bluetooth He..	OK
10:31:18	ABCD	RFID	—	Unknown

Warning — DUPLICATE_RFID_SCAN

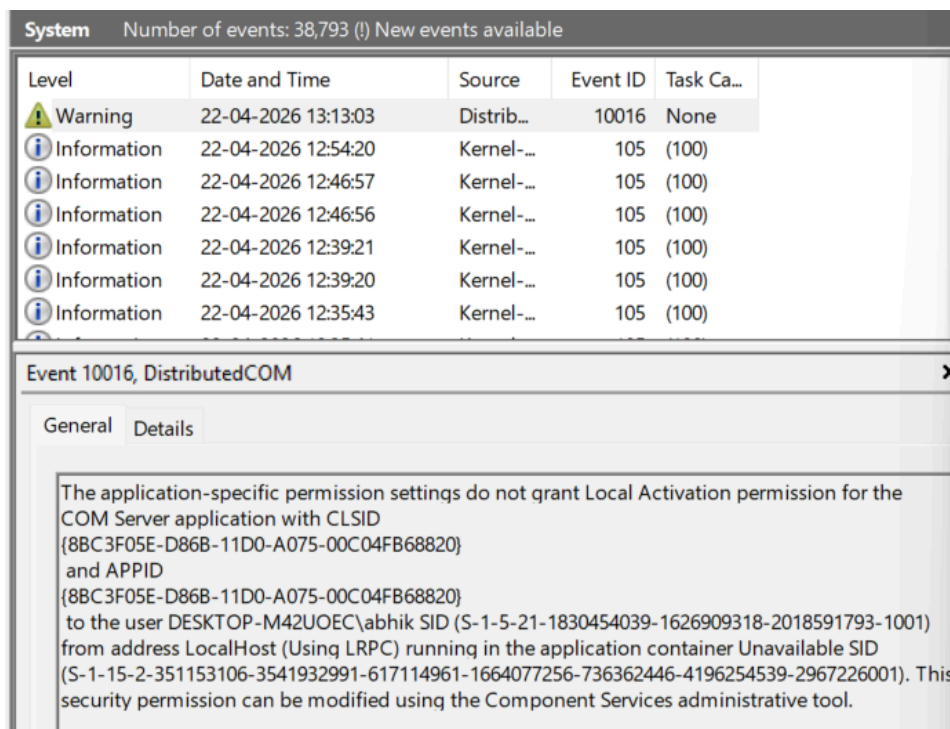
RFID tag 'RFID-0002' has already been scanned at this checkpoint. Duplicate discarded; staff alerted.

OK

Exceptions UI:

Exception Log "Barcode/RFID"					
Exception...	Exceptio...	Severity	Subsyst...	Error Me...	Timestamp
409	DUPLICA...	WARNING	Barcode/R...	Same RFI...	2026-04-22 22:37:30.026
408	UNKNOW...	WARNING	Barcode/R...	Scanned R...	2026-04-22 22:35:41.730
408	UNKNOW...	WARNING	Barcode/R...	Scanned R...	2026-04-22 13:15:24.946
408	UNKNOW...	WARNING	Barcode/R...	Scanned R...	2026-04-22 13:07:13.550
408	UNKNOW...	WARNING	Barcode/R...	Scanned R...	2026-04-22 12:59:15.412
409	DUPLICA...	WARNING	Barcode/R...	Same RFI...	2026-04-22 10:38:36.342
409	DUPLICA...	WARNING	Barcode/R...	Same RFI...	2026-04-22 10:36:20.879
408	UNKNOW...	WARNING	Barcode/R...	Scanned R...	2026-04-22 10:31:17.754

Event Viewer: The exceptions results were not visible on the event viewer because of Anti-Virus



Individual contributions of the team members:

Name	Module worked on
Abhikruti GNS	Database Manager (Deals with how the methods interact with the database internally), Rfid Exception (Deals with how and where the exceptions defined and called as pop up), Integrations with Other subsystems (Warehouse, database and exception)
Adithi Prabhu	Tag validator, Scan method, scan recorder (behavioral Patterns in the subsystem stem from here), Data class product and scan record
Anusha	RfidSystemFacade (coordinator of the entire RFID subsystem and the bridge between other subsystems and rfid subsystem), CSV exporter and logger
Ananya J C	Complete UI of the system (includes the main app window, the log panel, and the UI dialog helper (centralizes all popup dialogs for consistent UI presentation).))